UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Red Social UPT

Curso: Patrones de Software

Docente: Mag. Ing. Patrick Cuadros Quiroga

Integrantes:

Cutipa Gutierrez, Ricardo (2021069827)

Málaga Espinoza, Ivan (2021071086)

Chino Rivera, Angel (2021069830)

Tacna – Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	IME	CCC	CCC	01/062025	Versión Original

INDICE GENERAL

1. Antecedentes 1
2. Planteamiento del Problema 4
   a. Problema
   b. Justificación
   c. Alcance
3. Objetivos 6
4. Marco Teórico
5. Desarrollo de la Solución 9
   a. Análisis de Factibilidad (técnico, económica, operativa, social, legal, ambiental)
   b. Tecnología de Desarrollo
   c. Metodología de implementación (Documento de VISION, SRS, SAD)
6. Cronograma 11
7. Presupuesto 12
8. Conclusiones 13

Recomendaciones 14

Bibliografía 15

Anexos 16

Anexo 01 Informe de Factiblidad

Anex0 02 Documento de Visión

Anexo 03 Documento SRS

Anexo 04 Documento SAD

1) Antecedentes

La Escuela Profesional de Ingeniería de Sistemas de la Universidad Privada de Tacna (UPT) promueve la formación integral mediante la aplicación de patrones de diseño y arquitecturas modernas en el desarrollo de software. En este contexto, surge la necesidad de una plataforma de comunicación que resuelva la fragmentación de la información académica. Históricamente, la comunidad universitaria ha dependido de herramientas externas como WhatsApp o Facebook, las cuales carecen de una identidad institucional verificada y no ofrecen una segmentación por facultades, lo que motivó la creación de la Red Social UPT como una solución centralizada y segura.

2) Planteamiento del Problema

a) Problema

El problema central radica en la dispersión de la comunicación estudiantil en múltiples plataformas no institucionales. Esto genera falta de privacidad, ausencia de canales de comunicación segmentados por carrera y riesgos de seguridad al no poder verificar la identidad institucional de todos los participantes. Además, no existe un repositorio común para el intercambio de noticias y eventos exclusivos de la UPT.

b) Justificación

La implementación de este sistema se justifica por la necesidad de fortalecer la identidad institucional y garantizar un entorno digital controlado. Al utilizar una arquitectura de microservicios con PHP/Lumen, el sistema ofrece la escalabilidad necesaria para crecer junto con la comunidad universitaria, optimizando la gestión de interacciones y liberando a los estudiantes de la dependencia de redes sociales comerciales para fines académicos.

c) Alcance

El proyecto abarca los siguientes módulos funcionales implementados en la primera unidad:

- Autenticación Institucional: Registro e inicio de sesión exclusivo mediante Google OAuth para el dominio @virtual.upt.pe.
- Feed Dinámico: Muro de noticias con controles de visibilidad (Público, Compañeros, Facultad).
- Interacciones Sociales: Sistema de likes y comentarios en tiempo real.
- Gestión Social: Directorio de estudiantes y sistema de solicitudes de compañero.
- Mensajería Privada: Chat en tiempo real entre compañeros mediante un microservicio dedicado.
- Panel de Administración: Moderación de contenido y gestión académica de cuentas de usuario.

3) Objetivos

Objetivo General:

Desarrollar e implementar una plataforma web de red social universitaria basada en arquitectura de microservicios para la comunidad de la UPT, mejorando la seguridad, la trazabilidad de la información y la interacción académica.

Objetivos Específicos:

- Diseñar una arquitectura escalable de microservicios utilizando contenedores Docker y orquestación con Docker Compose.
- Garantizar la identidad institucional mediante la integración de Google OAuth 2.0.
- Implementar un sistema de comunicación segura inter-servicios basado en tokens JWT.
- Asegurar la calidad y seguridad del código mediante el uso de SonarCloud y Snyk en un pipeline de CI/CD.

4) Marco Teórico

- Microservicios: Arquitectura que divide la aplicación en servicios independientes comunicados por APIs REST.
- Lumen: Micro-framework de PHP optimizado para el rendimiento en servicios backend.
- JWT (JSON Web Token): Estándar para la transmisión segura de información de autenticación entre servicios.
- Infraestructura como Código (IaC): Práctica de gestionar servidores mediante archivos de configuración (Terraform).

5) Desarrollo de Solución

a. Análisis de Factibilidad

i. Factibilidad Técnica: El equipo cuenta con la experiencia en PHP y entornos Linux necesarios para el despliegue en una VPS Debian.

ii. Factibilidad Económica: El costo total del proyecto se estima en S/ 4565.00, incluyendo personal referencial y costos de nube.

iii. Factibilidad Operativa: El sistema es viable ya que utiliza las cuentas de Google Workspace ya existentes en la universidad.

b. Tecnología de Desarrollo

Herramienta	Especificación
Lenguaje	PHP 8.2 (Lumen)
Base de Datos	MySQL 8.0
Base de Datos	Docker & Docker Compose
Infraestructura	Terraform & VPS Debian

c. Metodología de Implementación

Se adoptó el Proceso Unificado de Racional (RUP) adaptado a un ciclo de desarrollo de 25 días, cumpliendo con las fases de Inicio (Alcance), Elaboración (Documentación FD01-FD03) y Construcción (Desarrollo de microservicios).

6) Cronograma

El proyecto se desarrolló durante 4 semanas del semestre 2026-I, culminando el 25 de abril con la entrega de la unidad 1, abarcando desde el análisis de requerimientos hasta el despliegue en la nube.

7) Presupuesto

Categoría	**Costo (S/)**
Personal (3 Desarrolladores)	3600.00
Infraestructura y Operativos	870.00
Gastos Generales	95.00
Total	4565.00

8) Conclusiones

- **Centralización y Autenticación Institucional Robusta:** Se ha logrado consolidar una plataforma digital que resuelve de raíz el problema de la dispersión de la información académica en la comunidad de la UPT. Al delegar la autenticación a Google OAuth 2.0 e implementar una validación estricta del dominio @virtual.upt.pe, el sistema no solo garantiza que cada interacción provenga de un miembro legítimo de la universidad, sino que también elimina la carga de gestionar credenciales sensibles en bases de datos propias, fortaleciendo la confianza del usuario final.

- **Escalabilidad mediante Microservicios e Independencia Funcional:** La transición de una arquitectura monolítica hacia una basada en microservicios con PHP/Lumen ha sido un éxito técnico. La separación de los dominios lógicos de Autenticación, Publicaciones, Relaciones Sociales y Chat permite que cada componente opere de forma autónoma con su propia base de datos MySQL, facilitando el mantenimiento preventivo y permitiendo que picos de tráfico en un servicio (como el Chat) no degraden el rendimiento de otros módulos críticos como el Feed.

- **Excelencia Técnica en la Aplicación de Patrones de Software:** El cumplimiento de los objetivos académicos se evidencia en la implementación del patrón Service Layer (Capa de Servicio) en todos los servicios backend. Al desacoplar la lógica de negocio de los controladores, el código resultante es significativamente más limpio, modular y fácil de someter a pruebas unitarias. Esta estructura permite que el sistema cumpla con los principios de ingeniería de software modernos y facilita la integración de futuras funcionalidades sin incurrir en una deuda técnica excesiva.

- **Infraestructura como Código y Despliegue Seguro:** La adopción de Terraform para la provisión de la VPS en Hetzner y Docker Compose para la orquestación ha transformado el despliegue en un proceso totalmente reproducible y automatizado. Además, la integración de un pipeline de CI/CD con SonarCloud y Snyk ha permitido que el código sea auditado automáticamente en cada cambio, asegurando que la plataforma esté libre de vulnerabilidades críticas y cumpla con los estándares de calidad exigidos por la cátedra.

9) Recomendaciones

- **Optimización del Tiempo Real mediante WebSockets:** Se recomienda que para la siguiente fase de evolución del producto, el microservicio de Chat migre del mecanismo actual de polling (consultas cada 3 segundos) hacia una arquitectura basada en eventos utilizando WebSockets (como Ratchet o Laravel Reverb). Esto reduciría drásticamente el consumo innecesario de recursos en el servidor y mejoraría la experiencia de usuario al permitir una comunicación bidireccional instantánea sin la latencia inherente al protocolo HTTP tradicional.

- **Ampliación de la Estrategia de Pruebas de Integración:** Dado que la comunicación entre servicios depende críticamente de la validez de los tokens JWT y el secreto compartido (JWT_SECRET), es fundamental implementar una suite de pruebas de integración automatizadas. Estas pruebas deben validar que los cambios en un contrato de API en el servicio de Auth no interrumpan el flujo de datos en los servicios de Posts o Social, garantizando la estabilidad del ecosistema ante actualizaciones futuras.

- **Monitoreo Proactivo de Recursos y Almacenamiento:** El sistema permite a los usuarios subir imágenes en publicaciones y chats, lo cual puede agotar rápidamente el almacenamiento SSD de la VPS. Se recomienda configurar alertas de monitoreo (como Prometheus o Grafana) para supervisar el uso de CPU, RAM y disco. Asimismo, se sugiere evaluar la integración de un servicio de almacenamiento de objetos (como AWS S3 o MinIO) para externalizar los archivos multimedia y no depender exclusivamente del almacenamiento local del contenedor.

- **Mantenimiento de la Wiki y el Roadmap de Producto:** Es imperativo continuar con la actualización constante de la Wiki de GitHub, documentando no solo las características actuales sino también el registro de cambios (changelog) de cada release. Esto servirá como una base de conocimientos técnica invaluable para los nuevos integrantes del equipo o para las auditorías académicas, asegurando que la visión del producto a largo plazo se mantenga alineada con las necesidades cambiantes de la comunidad universitaria de la UPT.

BIBLIOGRAFIA

1. Nishal Gurung, Sushil Shrestha, Rajani Chulyadyo (2025). "Scalability in Microservices: A systematic literature review". Journal of Computer Science and Technology, 25(2), 128-145. DOI: 10.24215/16666038.25.e11..
2. Nikesh Bahadur Adhikari (2024). "Evaluating security tools in the context of DevSecOps". University of Tampere, Facultad de Tecnología de la Información. Link: https://trepo.tuni.fi/handle/10024/156209..
3. Abhiram Reddy Peddireddy (2024). "Terraform-Driven Kubernetes Cluster Management in AWS". Journal of Artificial Intelligence, Machine Learning and Data Science, 2(1), 1-7. DOI: 10.51219/JAIMLD/abhiram-reddy-peddireddy/185..
4. Naga Murali Krishna Koneru (2025). "Infrastructure as Code (IaC) for Enterprise Applications: A Comparative Study of Terraform and CloudFormation". American Journal of Technology, 4(1), 10-25. DOI: 10.58425/ajt.v4i1.351..
5. Alexander Sliusarchyn (2023). "Service Layer in Laravel — use it!". Medium: Modern Software Architecture Series. Link: https://medium.com/@sliusarchyn/service-layer-in-laravel-use-it-ae861fb0f124..
6. Florian Farke, et al. (2023). "On the Security of Modern OAuth 2.0 Implementations: Vulnerabilities, Attacks, and Mitigations". European Symposium on Research in Computer Security (ESORICS), 1-20. DOI: 10.1007/978-3-031-51478-4..
7. Google Cloud Architecture Center (2023). "Cloud SQL Architecture Patterns for Microservices". Google Technical Whitepapers. Link: https://services.google.com/fh/files/misc/microservices_on_cloudsql_whitepaper.pdf..
8. Hasan Chinthaka (2024). "Stop Writing Messy Laravel Code: Use These Design Patterns". Tech Insights Report. Link: https://medium.com/@chinthakahasan/stop-writing-messy-laravel-code-use-these-design-patterns-90f61aef5185..

9. Atharva Shah (2025). "How to Secure Microservices in Multi-Cloud Architecture". AccuKnox Cybersecurity Blog. Link: https://accuknox.com/blog/microservice-security..
10. Nishal Gurung, et al. (2025). "Trends and Best Practices in API-Based Web Development Using Laravel and React". ResearchGate Preprint. DOI: 10.13140/RG.2.2.14567.8901.